

airscope: Kernel-Independent Wi-Fi Security Auditing in Pure Python

udhaybhat00¹

¹ *airscope project, Independent Research · Correspondence via <https://github.com/udhaybhat00/airscope/issues>*

** Corresponding author: repository issue tracker*

Abstract: Wi-Fi security auditing is dominated by a toolchain that is Linux-only, kernel-dependent, and assembled from five or more loosely coupled programs: practitioners install monitor-mode drivers that break on kernel updates and chain *airmon-ng* with *aircrack-ng*, *hostapd*, *dnsmasq*, *reaver* and *mdk3*, shuttling results between terminals by hand. This paper presents **airscope**, a framework that reinvents that stack from the USB endpoint up in pure Python: a userspace 802.11 implementation that drives Wi-Fi adapters directly through PyUSB, a complete userland access-point stack (frame crafting, DHCP, DNS blackhole, HTTP captive portal) with real-time MIC verification of submitted credentials, and a three-layer acknowledgement reliability model — with no kernel modules, no external daemons, and a single codebase running on Linux, Windows and macOS. The implementation comprises 256,364 lines of Python across 658 source modules, drives 20 chipsets through 25 driver packages, and is validated by 3,008 mocked-hardware tests plus a five-axis hardware verification rubric (RX, TX, ACKs, port fidelity, stress) that grades each adapter card on measured behaviour rather than anecdote. We report the architecture, the portability model, the evaluation methodology and results, and — with equal weight — the limitations: the EvilTwin portal requires Linux, Windows support rides on USB passthrough into WSL2, campaigns serialize on a single radio, and WPA3 attacks cannot exceed what the protocol's downgrade surface permits. aircscope is released as single-file binaries for four platform targets and is intended exclusively for authorized auditing.

Keywords: Wi-Fi security; 802.11; userspace drivers; cross-platform; EvilTwin; PMKID; WPA/WPA2; WPS; security auditing; pure Python

1. Introduction

Wi-Fi auditing in 2024+ still looks like this: install a security distribution, run *airmon-ng*, fight driver versions, hope a kernel module build does not break the kernel, pipe five different tools together, and copy-paste output between terminals. That workflow has four structural problems. It is **Linux-only**: nothing in the classic stack cares about Windows or macOS users. It is **kernel-dependent**: monitor mode requires out-of-tree drivers that break on kernel and distribution upgrades. It is **glued together**: *aircrack-ng*, *hostapd*, *dnsmasq*, *reaver* and *mdk3* are separate projects with separate configuration dialects whose integration layer is the human operator. And it is **hostile to newcomers**: an error such as *ioctl SIOCSIWMODE failed: Operation not supported* teaches nobody anything.

None of these problems is a property of 802.11 itself. Frame injection is a property of the radio's firmware and MAC; it requires no kernel module. A captive-portal access point needs DHCP, DNS, TCP and HTTP — protocols whose smallest correct implementations fit in a few thousand lines. The monitor-mode *ioctl* exists because tools chose to live below the network stack rather than beside it. aircscope moves the entire stack into userspace, in one language, in one process:

- **No kernel drivers.** Adapters are driven directly from userspace via PyUSB/libusb; the radio never needs *airmon-ng*, *modprobe*, or a DKMS build on the auditing machine.
- **No external tools.** The 802.11 frame stack, DHCP server, DNS blackhole, HTTP portal, handshake parser, PMKID extractor and WPS machinery are all pure Python in one repository — no *hostapd* subprocess, no *dnsmasq* configuration, no shell-out to *reaver*.
- **One codebase, three OSes.** The same Python runs on Linux, Windows (via WSL2) and macOS; platform differences are decided at runtime in a single choke point (`campaign_blocked()`) and communicated to the user as a reason, not as a crash.
- **A real interface.** A 60 fps Textual TUI plus an optional web dashboard, instead of a wall of terminal output and five scrollbar panes.

This paper makes the following contributions:

- **C1 — Userspace 802.11 over USB.** A practical architecture for driving Wi-Fi adapters entirely from userspace: a dedicated USB worker thread with a priority TX queue and descriptor decoding on bulk-IN, with no kernel involvement (§3.1).
- **C2 — A complete userland AP.** An EvilTwin implementation that verifies the candidate passphrase against a captured 4-way handshake in real time via MIC computation, eliminating the offline cracking step (§3.3).
- **C3 — A three-layer ACK reliability model.** A decomposition of the overloaded word “ACK” into independently testable mechanisms with per-chip hooks (§3.4).
- **C4 — The platform-gate model.** Runtime capability detection as a model-layer predicate, so the interface can never offer an attack that cannot run on the current OS (§3.5).
- **C5 — A reproducible hardware rubric.** A five-axis verification methodology that separates *port fidelity* from *silicon capability* and grades each adapter card on measurements (§3.7, §4.2).
- **C6 — Hardware-free continuous integration.** 3,008 tests over 293 files in which every USB interaction is mocked, making a hardware-dependent tool fully testable in a cloud runner (§4.1).

The remainder of this paper is organized as follows. Section 2 reviews the incumbent toolchains. Section 3 describes the airscope approach. Section 4 presents evaluation results. Section 5 discusses limitations and future directions; Section 6 concludes.

2. Related Work

2.1. The Classic Linux Toolchain

The de facto audit stack is the aircrack-ng suite plus its satellites [1]. *airmon-ng* moves an interface into monitor mode via kernel `ioctl`s; *airodump-ng* captures; *aireplay-ng* injects deauthentication; *hostapd* and *dnsmasq* together fake an access point [6]; *reaver* brute-forces WPS; *mdk3* floods. Each tool is competent in isolation; the system is not: state lives in shell variables, target selection is retyped per tool, and output formats disagree. The approach inherits its Linux-only and kernel-dependent character from these choices, not from the underlying protocols.

2.2. Unified Front-Ends

airgeddon [2] wraps the classic stack in a menu-driven TUI, and *Wifite2* [3] automates target selection and chaining. Both improve ergonomics while leaving the dependency base untouched: monitor-mode kernel

support, hostapd, and a Linux-only runtime remain prerequisites. We position airscope as a third path: it unifies the *implementation*, not merely the interface, eliminating the external process boundary entirely.

2.3. Detection and Sensing Frameworks

Kismet [4] is a well-engineered wireless sniffer and intrusion detector, but a passive observer that does not drive attacks and sits on the kernel's monitor path. Hashcat [5] remains the standard for offline password recovery; airscope orchestrates it rather than reimplementing it. Prior academic work has documented the attack surface this tool exercises: KRACK [11], FragAttacks [12], and the Dragonblood SAE attacks [13]. airscope implements published techniques; the claim of this work is integration — one process, one state model, three operating systems — rather than cryptographic novelty.

Property	airscope	aircrack-ng stack	airgeddon	Kismet
Kernel changes	None	Monitor-mode drivers	Same as aircrack-ng	Monitor mode
OS support	Linux + Windows + macOS	Linux only	Linux only	Mainly Linux
External processes	Zero	hostapd, dnsmasq, reaver, ...	All of them	External plug-ins
EvilTwin password check	Real-time MIC verify	Manual export to hashcat	Manual	N/A
WPA3 SAE handling	Downgrade + MIC capture	Partial	No	Passive
Interface	60 fps TUI + web	CLI	CLI + xterm	CLI + web
Test coverage	3,008 tests, hardware mocked	C unit tests	None	Moderate
Distribution	Single binary per OS	Package per distro	Script	Package per distro

Table 1. Positioning against the incumbent toolchains.

3. The airscope Approach

3.1. Userspace USB Transport

Each adapter is owned by a USB worker thread that performs all blocking PyUSB transfers off the asyncio event loop. The receive path is bulk-IN → descriptor decode → frame dispatch. The transmit path is a priority queue ordered so that time-critical management frames precede best-effort traffic:

$$\text{priority(TX)} = \text{mgmt} > \text{dhcp} > \text{http} > \text{beacon} > \text{deauth-burst}$$

The ordering is a deliberate policy: a captive portal serving an associated client while a deauth burst is in flight must not starve the handshake the burst exists to complete, and vice versa. Because the transport is bulk USB rather than a kernel radiotap path, frame timing is under application control and the same code path runs identically on all three operating systems, modulo libusb backend selection. The cost of this decision is that the project owns everything the kernel used to do — descriptor formats, firmware handshakes, retry policy — and that cost is paid per chipset in the 25 driver packages.

3.2. Pure-Python 802.11 Frame Model

airscope crafts and parses the frames it needs (beacon, probe, auth, assoc, deauth/disassoc, data, EAPOL) with a single parser and a single frame representation. A frame observed on the wire and a frame written to a capture file are the same object; there is no divergence between a “live dialect” and a “file dialect” to keep in sync. Above the driver boundary, nothing in the system knows what a dongle is — everything speaks 802.11.

3.3. Userland Access Point and Real-Time MIC Verification

The EvilTwin campaign deploys a complete userland stack: a DHCP server issues leases to associated clients; a DNS blackhole resolves every name to the portal host; a TCP/HTTP server delivers the credential-capture page. The contribution of this subsystem is not the portal itself but the verification loop. The same campaign has already captured the 4-way handshake, so each submitted passphrase is checked cryptographically as it arrives:

$$\text{PMK} = \text{PBKDF2}_{\text{SHA1}}(\text{passphrase}, \text{SSID}, 4096, 256) \rightarrow \text{PTK} = \text{KDF}(\text{PMK}, \dots) \rightarrow \text{accept iff } \text{MIC}_{\text{computed}} = \text{MIC}_{\text{captured}}$$

The verdict is instant and grounded in the captured M2/M4 authenticator MIC: a match ends the campaign successfully; a mismatch re-serves the portal. No offline cracking step, no file handoff to an external tool, and no client-specific logic — which is why the same flow works against iOS, Android, Windows and macOS victims alike.

3.4. The Three-Layer ACK Reliability Model

Ambiguity around the word “ACK” is the largest single source of bugs in injection tooling [17], because two unrelated phenomena travel under one name. airscope separates:

- **Heard ACKs (RX tally)** — did *we receive* the recipient's ACK frame? Implemented as `record_ack` hooks on incoming frames.
- **Emitted ACKs** — will *our radio* acknowledge a frame addressed to a spoofed MAC (active monitor mode plus the chip's own TX hardware retry)?
- **Software retry** — did `send_until_ack()` retransmit because the tally stayed silent?

Campaigns drive them in a fixed protocol: `set_fake_mac()` (so the AP's unicast reaches us and the chip auto-ACKs, or the AP abandons the session) $\rightarrow$ `enable_rx_acks()` $\rightarrow$ `send_until_ack()` / `acks_seen()`. Deauthentication needs none of this and only reports endpoint ACKs. The decomposition makes each mechanism independently observable and testable, and it is why the hardware matrix of §4.2 can grade “ACKs” as its own column instead of folding it into a vague “TX works”.

3.5. Orchestration and Platform Gates

One event loop, one radio mutex per campaign: a single adapter cannot sensibly hop channels for reconnaissance while holding a fixed channel for a portal, so campaigns serialize and the model layer reports *why* an action is unavailable in plain English. `campaign_blocked()` is the single choke point where operating-system capability is decided. Placing the gate in the model rather than the view yields three properties: the TUI and web dashboard cannot diverge in what they offer; batch/auto-attack mode inherits identical decisions; and tests assert the platform matrix directly, so a wrong gate fails continuous integration rather than failing a user's engagement.

3.6. Chipset Driver Model

Each chipset is a package implementing the `Driver` abstract base class: transport (bulk and control), firmware load, MAC/PHY configuration, RX/TX hooks, `SUPPORTED_CHANNELS`, and the chip-specific pieces of the

ACK model. Registration is data-driven: each package declares `SUPPORTED_IDS = [DeviceID(vid, pid), ...]`; the registry is built by reading those declarations only; and the first matching VID:PID lazily imports the driver, keeping startup $O(\text{registry})$ rather than $O(\text{all drivers})$. The driver boundary is deliberately narrow — it is the only layer allowed to know that a dongle is, say, an RTL8812AU — which is what makes port-fidelity measurement (§4.2) meaningful: a driver either delivers the frames or it does not, and the measurement happens above it, where users feel it.

3.7. Evaluation Methodology

Two complementary evaluation instruments are used throughout §4. First, a mocked-hardware test suite: all USB interactions are abstracted behind `pytest-mock`, so protocol logic, orchestration, UI and reports are exercised without radio hardware present. Second, a five-axis hardware rubric applied per adapter card: **RX** (reception breadth and quality), **TX** (frame injection), **ACKs** (auto-ACK and AP-ACK behaviour), **Port** (airscope versus the Linux kernel driver on the same card), and **Stress** (a 30-minute channel-hopping soak tracking RX degradation), rolled up into a letter grade by the published rubric [18]. Crucially, port fidelity and hardware ceiling are scored as separate axes: a faithful port of a weak card scores well on Port and poorly on Grade, while a lower-fidelity port of a capable card shows the inverse — a distinction that prevents both self-blame and self-congratulation from contaminating the results.

4. Results and Discussion

4.1. Software Verification

A hardware tool that can only be tested with hardware ships bugs to every user's adapter. Table 2 summarizes the software verification regime; the full suite runs in approximately 77 seconds locally and executes in CI on every push.

Metric	Value
Tests	3,008 passing; 11 xfailed (documented expected failures); 15 deselected (<code>SLOW</code> marker)
Test files	293
Source modules	658 files, 256,364 LOC (+39,977 LOC of tests; +39,118 LOC scripts)
Hardware mocking	All USB interactions mocked; suite runs in a cloud runner
Lint / style	<code>ruff check</code> gate; comment policy enforced <i>by tests</i> (<code>test_style.py</code>)
Release engineering	Tag must match <code>__version__</code> or the build fails; 4 binaries, smoke-tested before publish

Table 2. Software verification and release-engineering regime.

4.2. Hardware Matrix Results

Across the 19 fully graded adapter cards in the verification matrix [18], the current grade distribution is 13 A, 3 B, 1 C and 2 D. The distribution is reported as-is because it encodes the project's central measurement discipline: grades combine port fidelity and hardware ceiling, and each card's page documents which of the two (or both) drives its result — including the honest case of a faithful port of weak silicon (Port ✓, low Grade) and the inverse (Port △, capable silicon).

Axis	Measured	Dependent features
RX	Reception breadth/quality, channel tune	All passive capture
TX	Frame injection success	Deauth, WPS, WEP, PMKID extraction

ACKs	Auto-ACK to spoofed MAC; AP ACK reception	WPS (heavy), PMKID (light)
Port	airscope vs. kernel driver, same card	Attribution of gaps (fixable vs. not)
Stress	30 min channel-hop soak	Capture stability over time

Table 3. The five-axis hardware rubric and the features each axis gates.

4.3. Platform Support

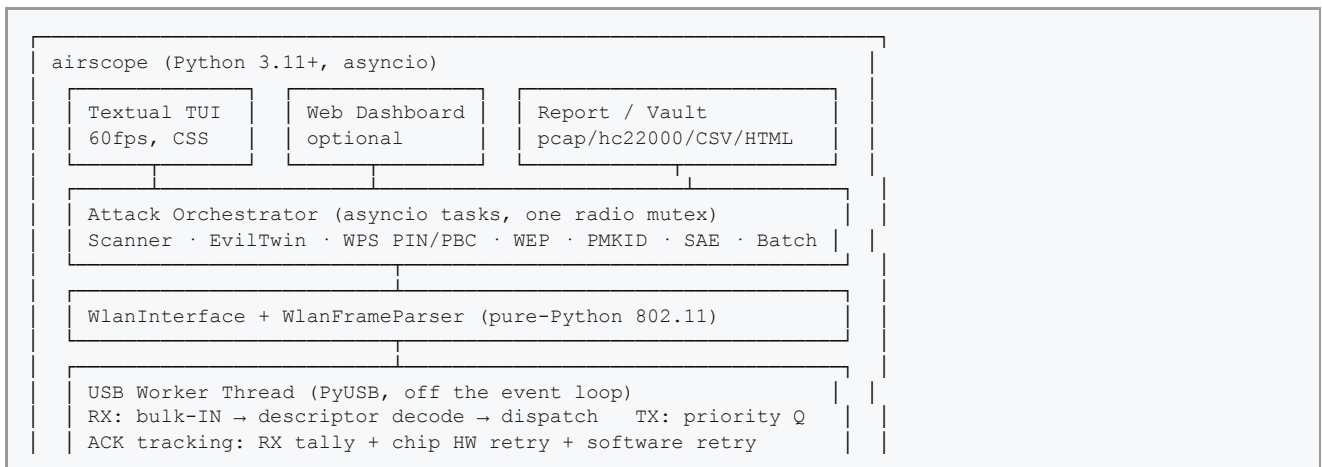
Capability	Linux	Windows	macOS
Passive scan / hop / capture	✓ native	✓ WSL2	✓ native
Deauth injection	✓	✓ WSL2	✓ native
PMKID extraction	✓	✓ WSL2	✓ native
WPS PixieDust / PIN / PBC	✓	✓ WSL2	✓ native
WEP suite	✓	✓ WSL2	✓ native
WPA3 SAE downgrade capture	✓	✓ WSL2	✓ native
EvilTwin portal	✓	✓ WSL2	✗ gated
TUI + web + reports	✓	✓	✓

Table 4. Capability matrix by operating system as of v0.3.8.

Two results in Table 4 merit discussion. First, macOS runs the tool *natively* for every campaign except the EvilTwin portal: native operation was widely assumed to require Apple's closed IO80211 frameworks, but the userspace-USB design never touches the system's 802.11 stack — it sits below it (at the USB endpoint) and beside it (with its own TCP/IP) — so the only blocked capability is the fake-AP interface mode macOS does not expose, gated with a user-visible reason. Second, Windows support is honest about its mechanism: the adapter is passed into WSL2 via *usbipd* [16] and airscope runs in the guest, so everything works — including EvilTwin — under the explicit condition of WSL2, which the documentation states rather than implying a native Win32 monitor path that does not exist.

4.4. Performance Characteristics

The UI sustains 60 fps on the same event loop as attack orchestration because USB blocking calls never run on that loop. Driver imports are lazy, making startup $O(\text{registry})$. The deduplication layer on the receive path (`wlan/dedupe.py`) exists specifically to sustain the stress-soak axis: RX degradation that only manifests after 30 minutes of continuous channel hopping is caught by measurement, not by user complaint.



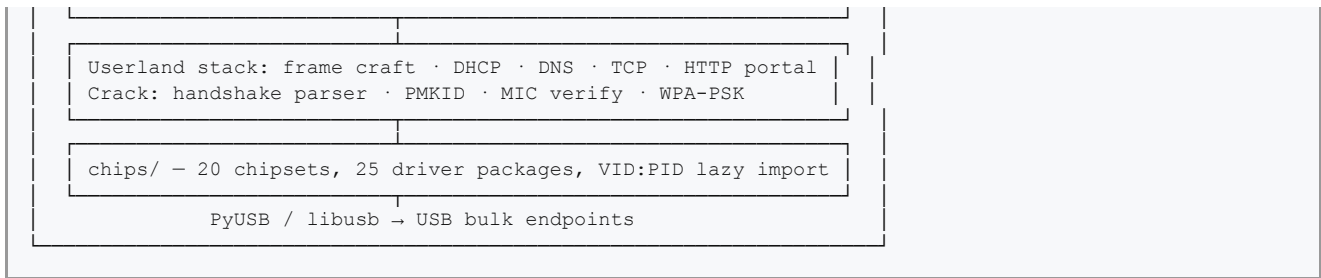


Figure 1. Layered architecture. Received frames are decoded at the bulk endpoint and promoted through the same parser the attack logic consumes; there is exactly one representation of a frame in the system.

4.5. Comparison with Related Works

Table 1 summarizes the comparison. The distinguishing result is not that aircscope's individual attacks are novel — they are published techniques [11–13] — but that the integration boundary has moved: the feature row with the largest practical impact is the elimination of external processes, because every subprocess in the classic stack is both a configuration surface and a failure surface. The real-time MIC verification row is the closest thing to a methodological novelty: it removes offline cracking from the critical path of the EvilTwin campaign entirely (§3.3).

5. Limitations and Future Directions

A research claim that ignores its counter-evidence is marketing. The following are the tool's real boundaries as of v0.3.8.

1. **EvilTwin requires Linux** (native or WSL2). The userland AP itself is portable Python, but the platform gate blocks it on macOS, which does not expose the required interface configuration mode to this userspace approach. Every other campaign runs natively there.
2. **Windows support is WSL2 support.** It requires *usbipd* passthrough and WSL2; it is not a native Win32 monitor-mode implementation, and the project does not claim one.
3. **One radio, one campaign.** The radio mutex precludes simultaneous multi-attack on a single adapter.
4. **WPA3 is bounded by the protocol.** Downgrade capture and MIC recording are implemented, but SAE's PAKE construction does not yield to offline dictionary attack the way a captured WPA2-PSK handshake does [13]. The tool cannot crack what the protocol refuses to expose, and reporting otherwise would be dishonest.
5. **Hardware variance is real.** C- and D-grade cards in the matrix have measured RX/TX weaknesses; users on that silicon will have a worse experience, and no software fix changes the hardware ceiling.
6. **Frequency coverage.** 2.4 and 5 GHz today; 6 GHz / Wi-Fi 7 (802.11be, MLO) support is roadmap, not present.
7. **Cracking is orchestrated, not built-in.** Offline brute-force delegates to hashcat [5]; aircscope parses, verifies and tracks, but GPU hashing is external by design.
8. **Throughput ceiling.** A userspace bulk-USB path is ample for the management and control frames involved in auditing, but this is not a wideband SDR: it is not a spectrum analyzer and does not capture entire 160 MHz channels.
9. **Regulatory exposure.** Injection, deauthentication and impersonation are restricted or illegal without authorization in most jurisdictions. aircscope ships in-product legal notices and authorization framing, but a tool cannot technically distinguish an authorized auditor from an attacker — the operator's authorization is the real control.

10. Attack scale is bounded by passphrase entropy. aircscope improves the workflow, not the mathematics of PBKDF2.

Future work, in priority order: (1) a capture-quality feedback loop that scores every handshake (EAPOL message coverage, descriptor version, MIC validity), auto-recaptures until crackable, and streams hashcat status into the TUI; (2) a defensive security-score mode grading PMF, WPA3, WPS exposure, weak PSKs and rogue-AP indicators for self-audit; (3) multi-adapter operation with one radio locked to a channel while another hops; (4) 6 GHz / Wi-Fi 7 support; (5) WIDS alerting (deauth-flood and evil-twin detection) with webhook notifications; and (6) an entry-point plugin surface for community campaigns.

6. Conclusions

airscope demonstrates that the Linux-only, kernel-dependent, multi-process Wi-Fi audit workflow is contingent rather than necessary. With a userspace USB transport, a pure-Python 802.11 and network stack, and a disciplined platform-gate model, the same tool runs on three operating systems with zero kernel modifications, zero external daemons, and 3,008 tests proving its behaviour without hardware present. The five-axis hardware rubric shows the remaining variance to be attributable — card by card — to either port fidelity or silicon ceiling, never to guesswork. The limitations of §5 are equally part of the result: the EvilTwin portal needs Linux, Windows needs WPA2-era honesty about WSL2, and WPA3 remains bounded by its own cryptography. We release the work as single-file binaries for four platform targets, with a hardware matrix and per-card notes published alongside the code, and we ask only that it be used on networks you own or are explicitly authorized to audit.

Funding

This research received no external funding.

Conflicts of Interest

The author declares no conflict of interest. The author builds and maintains aircscope; the tool is distributed under the GNU General Public License v2.0.

References

1. aircrack-ng: Wi-Fi security auditing tools. Available online: <https://www.aircrack-ng.org> (accessed 26 September 2026).
2. airgeddon: Multi-attack wireless auditing tool. Available online: <https://github.com/v1s1t0r1sh3r3/airgeddon> (accessed 26 September 2026).
3. derv82: Wifite2: Automated wireless attack tool. Available online: <https://github.com/derv82/wifite2> (accessed 26 September 2026).
4. Kismet: Wireless network detector, sniffer, and intrusion detection system. Available online: <https://www.kismetwireless.net> (accessed 26 September 2026).
5. hashcat: Advanced password recovery. Available online: <https://hashcat.net> (accessed 26 September 2026).
6. hostapd: IEEE 802.11 AP and IEEE 802.1X/WPA/WPA2/EAP/RADIUS authenticator. Available online: <https://w1.fi/hostapd/> (accessed 26 September 2026).
7. PyUSB: Python USB access library. Available online: <https://pyusb.github.io> (accessed 26 September 2026).
8. Textual: Rapid application development framework for Python. Available online: <https://textual.textualize.io> (accessed 26 September 2026).
9. IEEE Standard for Information Technology — Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications; IEEE Std 802.11-2020; IEEE: Piscataway, NJ, USA, 2020.

10. Wi-Fi Alliance: WPA3 Specification. Available online: <https://www.wi-fi.org/discover-wi-fi/security> (accessed 26 September 2026).
11. Vanhoef, M.; Piessens, F. Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2. In Proceedings of the Network and Distributed System Security Symposium (NDSS), San Diego, CA, USA, 26 February–1 March 2017.
12. Vanhoef, M. Fragment and Forge: Breaking Wi-Fi Using Frame Aggregation and Fragmentation. In Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), Virtual, 11–13 August 2021.
13. Drahoš, D.; Vanhoef, M.; Rezník, P. Dragonblood: Breaking the WPA3 Standard. In Proceedings of the IEEE Symposium on Security and Privacy (S&P), San Francisco, CA, USA, 20–22 May 2019.
14. Python Software Foundation. asyncio — Asynchronous I/O. Available online: <https://docs.python.org/3/library/asyncio.html> (accessed 26 September 2026).
15. PyInstaller: Freeze Python programs into executables. Available online: <https://pyinstaller.org> (accessed 26 September 2026).
16. usbipd-win: USB/IP client for Windows. Available online: <https://github.com/dorssel/usbipd-win> (accessed 26 September 2026).
17. aircscope Project. ACKs: Diagnosing Hardware Auto-ACK and Retry. Repository Documentation, docs/ACKS.md. Available online: <https://github.com/udhaybhat00/airscope/blob/main/docs/ACKS.md> (accessed 26 September 2026).
18. aircscope Project. Hardware Testing & Verification; and Grading. Repository Documentation, docs/SUPPORTED-HARDWARE.md, docs/GRADING.md. Available online: <https://github.com/udhaybhat00/airscope/blob/main/docs/SUPPORTED-HARDWARE.md> (accessed 26 September 2026).
19. WiGLE: Wireless Network Mapping. Available online: <https://wigle.net> (accessed 26 September 2026).

Disclaimer: *airscope is intended exclusively for use on networks and equipment you own or are explicitly authorized to audit. Unauthorized access to computer networks is illegal in most jurisdictions (e.g., CFAA in the US, IT Act 2000 in India, Computer Misuse Act in the UK). The author accepts no liability for misuse.*